

Route Graduation Project

AI Engineering Assessment — Intelligent RAG System

Submission Format: GitHub repository link + written report

Report Format: PDF or Markdown

Deadline: 2/6/2026

Overview

In this graduation route project, students will build an AI-powered question-answering assistant using a Retrieval-Augmented Generation system.

The system must allow a user to ask natural-language questions about a knowledge base created by the student. The assistant should retrieve relevant information from the collected data, pass the retrieved context to an LLM, and generate a grounded answer based only on the retrieved information.

Students are free to choose any public website or public web pages as their data source. The quality of the AI pipeline, retrieval logic, documentation, and explanation matters more than the selected topic.

Example domains may include:

- - Travel guides
 - • • • • FAQs
 - Product University
 - catalogues information
 - Medical Recipes
 - Technical documentation
 - • • News
 - Legal or articles
 - policy Educationa
 - pages l content

Part 0 — Dataset Collection & Chunking Strategy

0.1 — Choose and Collect Your Dataset by Scrapping as Last Session see the record and make the graduation project look like it

Students must collect their own dataset from any public website or public web pages.

In the report, students must describe:

- - The website or source used

- The URL or list of URLs
- The type of data collected
- The structure of the data
 - • • HTML pages
 - Plain text
- JSON
- CSV
- Tables
- Articles
- • records
- Product
 - The approximate number of pages, documents, or records collected
 - The method used to collect the data
 - The cleaning and preprocessing steps applied

Students must collect data responsibly and respect the website's terms, access rules, and public availability.

0.2 — Design Your Chunking Strategy

There is no predefined chunking method that students must follow. Each student must study their dataset first, then decide the best way to split it.

Students should think about:

- What is the natural unit of meaning in the data?
- A sentence?
- A paragraph?
- A section?
- A product record?
- A full page?
- Does splitting the text in the wrong place destroy the meaning?
- How will users probably ask questions about this data?
-

What chunk size gives the best chance of retrieving the correct answer?

The report must include:

- The selected chunking strategy
- Why this strategy was chosen
- What was observed in the data that influenced the decision
- What other strategies were considered and rejected
- Weaknesses of the chosen strategy
- How these weaknesses were reduced using chunk size, overlap, or metadata 2

Level 1 — Foundation RAG Pipeline

Goal

Build a working end-to-end RAG pipeline that can answer questions about the collected dataset.

1.1 — Data Ingestion

Students must:

- Load the collected dataset
- Clean and preprocess the text
- Apply the chosen chunking strategy
- Store metadata for each chunk

Metadata may include:

- Source URL
- . . . title
- Page
- Section title
-

Category	• • •	Record ID
	Date	
Any other useful information		

1.2 — Embedding & Indexing

Students must:

- Convert all chunks into embeddings
- Use an embedding model of their choice
- Store the embeddings in a vector store

Possible vector stores include:

	• •	FAISS
Chroma		

The report must justify the embedding model choice based on:

- • Language support
- Accuracy
- Speed
- Cost
- • dimension
- Embedding
- Suitability for the chosen dataset

3

1.3 — Basic Retrieval

Given a user query, the system must:

- Embed the query
- Retrieve the top-k most similar chunks
- Display the retrieved chunks
- Display similarity scores
- Display source metadata

The output should make it clear which documents or chunks were retrieved before the answer was generated.

1.4 — Answer Generation Using Gemini API

Students must use the Google Gemini API for answer generation.

The system must:

- Pass the retrieved chunks as context to Gemini
- Generate an answer grounded only in the retrieved context
- Avoid hallucinating facts that are not present in the retrieved chunks
- Display the final answer with the sources used

The prompt should clearly instruct the model to answer only from the provided context.

Level 1 Deliverable

A script or notebook that takes a user question as input and prints:

- The final answer
- The retrieved sources
- The retrieved chunks
- Similarity scores
- Metadata

Level 2 — Query Intelligence

Goal

Make the system smarter in understanding and handling user queries before retrieval.

The pipeline should follow this structure:

4

Original query → Query rewriting → Query classification → Filter extraction → Filtered retrieval → Answer generation

2.1 — Query Rewriting

Before retrieval, students must rewrite the user query to make it:

- Cleaner
- More specific
- Easier for the retrieval system to understand
- In English if the dataset is English or multilingual

Students may use Gemini API for query rewriting.

The output must show:

- Original user query
- Rewritten query
- Rewritten query

2.2 — Query Classification

Students must classify each incoming query into at least two categories relevant to their domain.

Example categories:

- Factual
- Comparison
- Recommendation
- General question
- Out of scope
- Price-related query
- Date-related query
- Location-related query
- Lookup

The query class should affect how the system handles the query. The

report must include at least one example for each query category.

2.3 — Structured Parameter Extraction

Students must extract important filters implied by the user query.

Example filters:

	Price range	Location
5		
		• • Date
Category		• • • Language
		Product genre
		type
Author		
•		
Topic		• y level
		Difficulty

The extracted filters should be represented as a structured object, such as:

	Python
• •	dictionary
	y
JSON object	
	• c model
	Pydantic

These filters should be used to narrow the search using metadata filtering where possible.

Level 2 Deliverable

An updated pipeline where each query goes through:

1.	Query rewriting
2.	Query classification
3.	Structured filter extraction
4. 5.	retrieval
Filtered	
	Gemini-based answer generation

Level 3 — Retrieval Quality: Hybrid Search

Goal

Improve retrieval quality by combining semantic search and keyword-based search.

3.1 — Research Hybrid Search

Students must search and study the concept of hybrid search by themselves.

They must understand and explain:

- What dense retrieval means
- What sparse retrieval means
- Why vector search alone may fail
- Why keyword search alone may fail
- How hybrid search combines both methods
- When hybrid search is useful

6

3.2 — Implement Hybrid Search

Students must implement hybrid search by combining:

- Dense retrieval using embeddings
- Sparse retrieval using keyword search or BM25

Students may combine the scores using:

- Weighted fusion
- Rank Fusion
- Reciprocal
- Any clearly explained custom scoring method

The implementation must be part of the student's own RAG pipeline.

3.3 — Compare Dense-Only vs Hybrid Search

Students must compare retrieval results for at least three different queries.

For each query, students must show:

- Top results from dense-only retrieval
-

Top results from hybrid retrieval

- Which one performed better
- Why the result improved or did not improve

The report must explain the difference in answer quality between dense-only retrieval and hybrid retrieval.

Level 3 Deliverable

A retrieval pipeline that supports hybrid search and includes a clear comparison between dense-only retrieval and hybrid retrieval.

Streamlit GUI Requirement

Instead of building a FastAPI inference API, students must build a simple Streamlit GUI for the project.

The Streamlit interface must include:

- Text input box for the user question
- Button to submit the question
- Final generated answer
- Retrieved source chunks
- Source metadata
- Similarity scores

7

- Original query
- . . . n query
- Rewritten query
- Query class
- Extracted filters, if available

The GUI should make the system easy to test and demonstrate.

Important rule:

Students must write the core RAG and LLM pipeline code themselves.
AI tools are allowed to generate only the Streamlit GUI code.

Written Report Requirement Students

must submit a written report explaining everything they built. The report must include:

1. Dataset Description

- Website or source used
- Source URLs
- Type of data
- Number of pages or records
- Final dataset format

2. Data Collection Method

- How the data was collected
- Tools or libraries used
- Cleaning steps
- Challenges faced during data collection

3. Chunking Strategy

- Chosen chunking method
- Reasoning behind the method
- Alternatives considered
- Weaknesses and mitigation

4. Embedding and Vector Store

- Embedding model used
- Why it was selected
- Vector store used
- Why it was selected

5. Gemini API Usage

- Where Gemini was used
 - . . . design
 - Prompt
- Grounding strategy
- How hallucination was reduced

6. Query Intelligence

- Query rewriting approach
- Query classification approach
- Filter extraction approach
- Examples for each query class

7. Hybrid Search

- Explanation of hybrid search
- Dense retrieval method
- Sparse retrieval method
 - . . . method
 - Fusion
- Comparison between dense-only and hybrid search
- Results for at least three queries

8. Streamlit GUI

- Screenshots of the GUI
- Explanation of interface components
- How the GUI connects to the pipeline

9. Limitations

Students should honestly discuss limitations such as:

-

- Small dataset size
- website structure
- Poor
- Weak metadata
- mistakes
- Retrieval
- Gemini limitations
- Query ambiguity

10. Future Work

Students may suggest improvements such as:

- Better scraping coverage
- Better metadata extraction
- More advanced chunking
- Better hybrid scoring
- More evaluation queries
- Deployment

- 9
- Authentication
- Chat history

Rules

- Python is strongly recommended.
- Students may use any embedding model.
- Students must use Google Gemini API for LLM-based generation.
- Students must build the core pipeline themselves.
- End-to-end tools that hide the implementation are not allowed.
- AI tools may be used for explanation, debugging help, and Streamlit GUI code only.
- The RAG pipeline, retrieval logic, chunking logic, Gemini prompts, and hybrid search implementation must be written by the student.
- Real API keys must never be committed to GitHub.

- A .env.example file must be included instead of real secrets.
- The report must explain design decisions, not only describe the final result.
- Academic integrity applies. Students should mention any AI tools used and explain how they were used.

Evaluation Rubric

Component Weight
Part 0: Dataset collection and chunking justification 10%
Level 1: Working RAG pipeline 25%
Level 2: Query intelligence 20%
Level 3: Hybrid search and retrieval quality 20%
Streamlit GUI 10%
Code quality, clarity, README, and report 15%
Total 100%

Submission Checklist

Students must submit:

- GitHub repository link
- Clear README file
- Setup and run instructions
- requirements.txt or pyproject.toml • .env.example file
- Source code for the full pipeline

10

- Streamlit GUI code
- Written report in PDF or Markdown
- Screenshots or short demo recording
- Example queries and outputs
- Dense-only vs hybrid search comparison

Final Deliverable

By the end of the project, students should have a working RAG application that can:

1. Collect and prepare data from a public website
2. Chunk and index the collected data
3. Retrieve relevant information using vector search
4. Improve retrieval using hybrid search
5. Use Gemini API to generate grounded answers
6. Apply query rewriting, classification, and filter extraction
7. Display the full system through a Streamlit GUI

Explain all design decisions in a written report 11